

Aesop tactic

Graduate Seminar on Applied Logic

Yannik Spitzley

June 6, 2026

Introduction

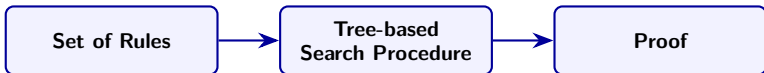
Idea behind Aesop

Aesop is an abbreviation:

Automated
Extensible
Search for
Obvious
Proofs

Examples:

- ▶ $2 \cdot n$ is even for $n \in \mathbb{N}$,
- ▶ $[z, y, x]$ is a permutation of $[x, y, z]$,
- ▶ $A \cap B \subseteq A$ for sets A, B ,
- ▶ $\vdots$



Motivating example

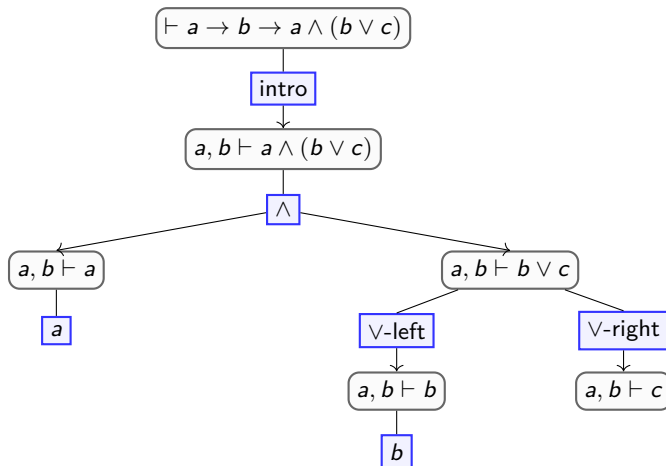
Exercise: Prove $a \rightarrow b \rightarrow a \wedge (b \vee c)$.

Manual proof in Lean:

```
example (a b c : Prop) : a → b → a ∧ (b ∨ c) := by
  intro ha hb
  constructor
  · exact ha
  · left
    exact hb
```

We can write this proof in a tree-like structure.

Motivating example



Aesop: White-Box Best-First Proof Search

White-box:

- ▶ Rule set (& other parameters) are user-specified.
- ▶ Interesting for domain-specific tools.
- ▶ Transparency: Easier to debug and manually close unfinished goals.

Best-First:

- ▶ Tree-based search is implemented with a best-first strategy.
- ▶ Nodes of the search tree are extended based on user-given priorities/settings.

Proof Search: Automated search for obvious proofs.

Best-First Proof Search

Goals and rules

Set of goals $\mathcal{G}$

Each goal has the form

$$\vec{h} : \vec{T} \vdash U$$

- ▶ $\vec{h}, \vec{T}$: Hypotheses
- ▶ U : Target

Set of rules $\mathcal{R}$

Each rule r is a partial function on $\mathcal{G}$:

$$r : g \mapsto G_{r,g} \subseteq \mathcal{G}$$

- ▶ r is **applicable** to g .
- ▶ $G_{r,g}$: Generated **subgoals**.
- ▶ **Provability-reflecting**:

$$G_{r,g} \text{ provable} \Rightarrow g \text{ provable}$$

- ▶ r **proves** g if $G_{r,g} = \emptyset$.

Example: The $\wedge$ -rule

Maps a goal $\Gamma \vdash P \wedge Q$ to the set $\{\Gamma \vdash P, \Gamma \vdash Q\}$.

Search tree

Alternating node structure:

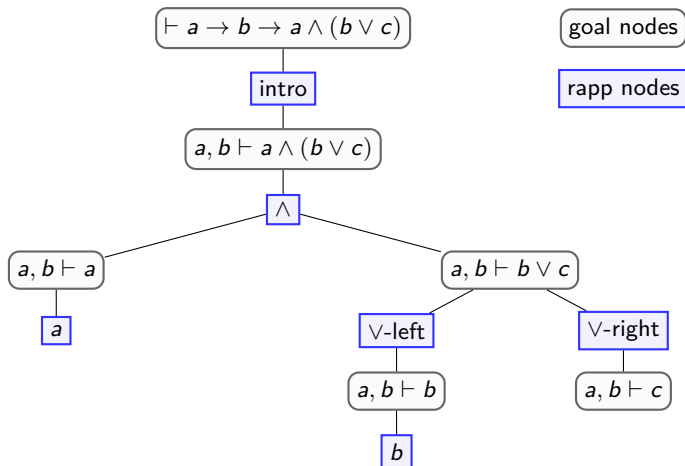
- ▶ **Goal nodes:** Children are rapp nodes for applicable rules.
- ▶ **Rule application (rapp) nodes:** Children are goal nodes for generated subgoals.

3 node states:

State	Goal node	Rapp node
proved	Has at least one <i>proved</i> child.	All children are <i>proved</i> .
stuck	Can not be <i>proved</i> with the given rules.	
unknown	Neither <i>proved</i> nor <i>stuck</i> .	

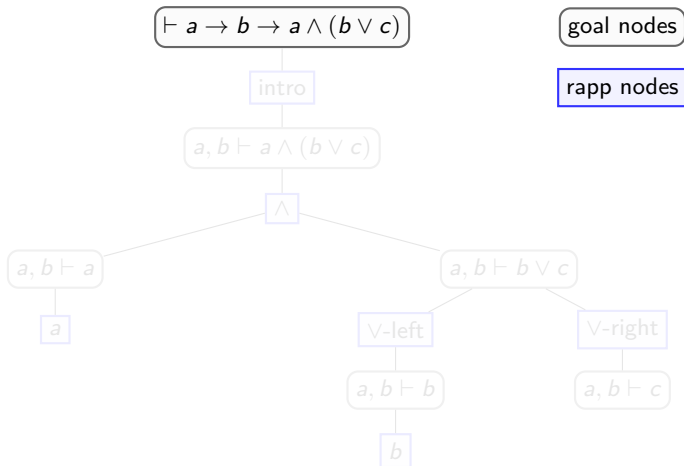
Search tree

proved
stuck
unknown



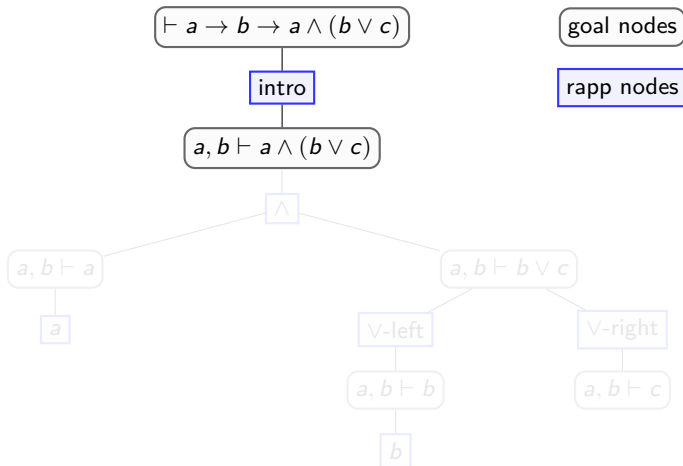
Search tree

proved
stuck
unknown



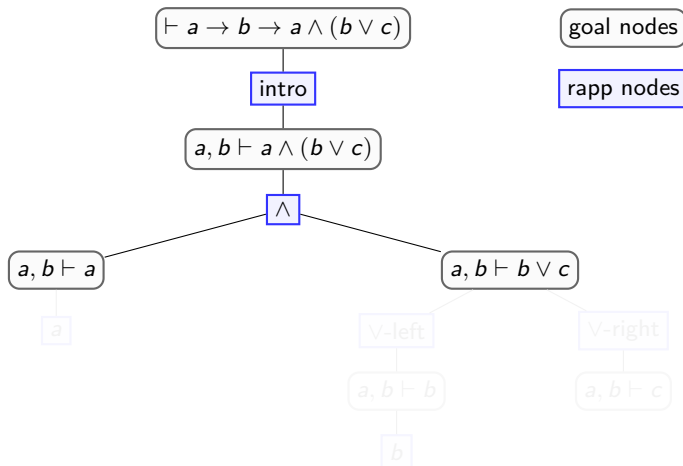
Search tree

proved
stuck
unknown



Search tree

proved
stuck
unknown

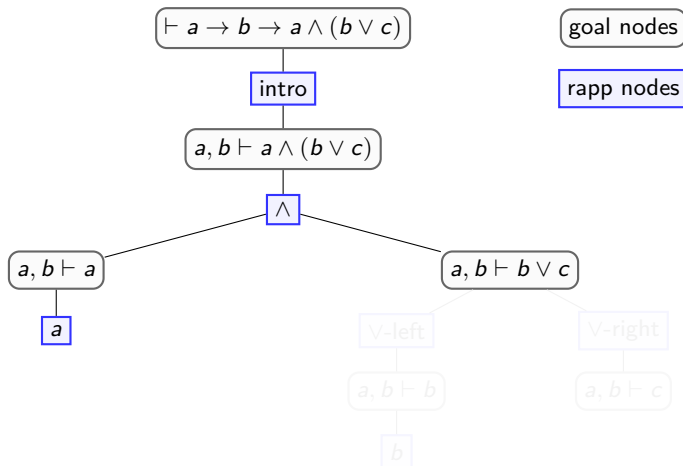


Search tree

proved
stuck
unknown

goal nodes

rapp nodes

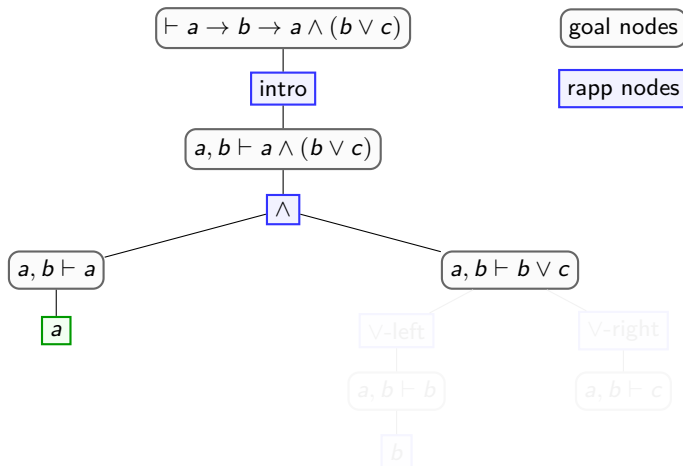


Search tree

proved
stuck
unknown

goal nodes

rapp nodes

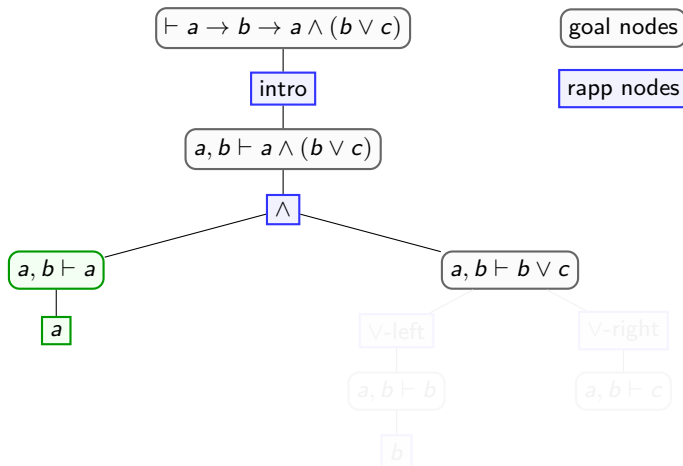


Search tree

proved
stuck
unknown

goal nodes

rapp nodes

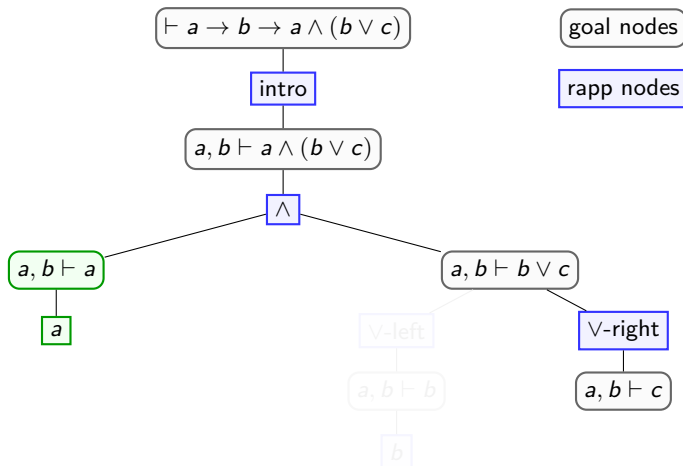


Search tree

proved
stuck
unknown

goal nodes

rapp nodes

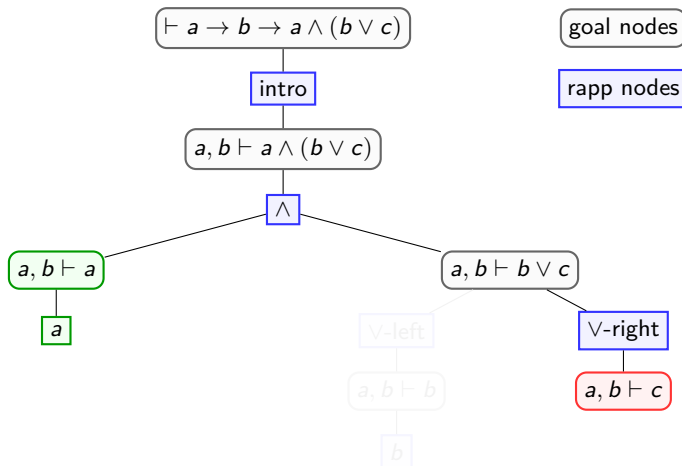


Search tree

proved
stuck
unknown

goal nodes

rapp nodes

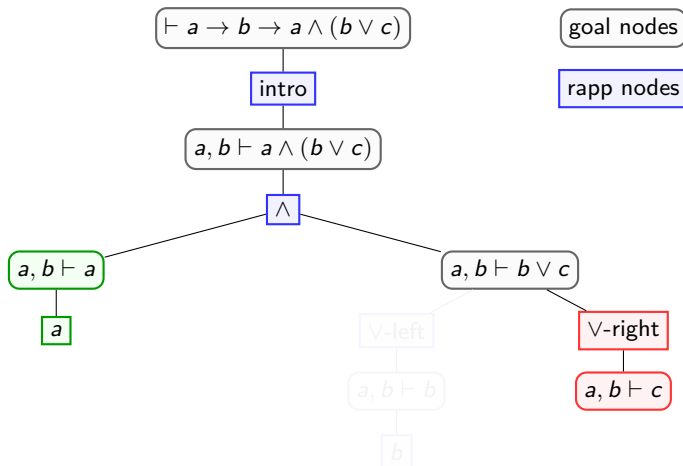


Search tree

proved
stuck
unknown

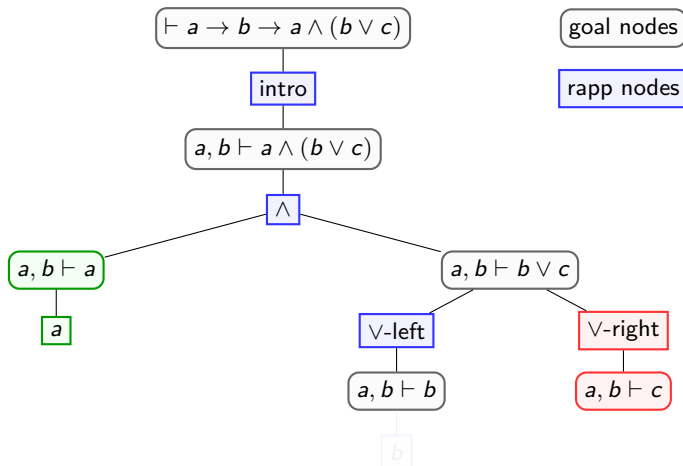
goal nodes

rapp nodes



Search tree

proved
stuck
unknown

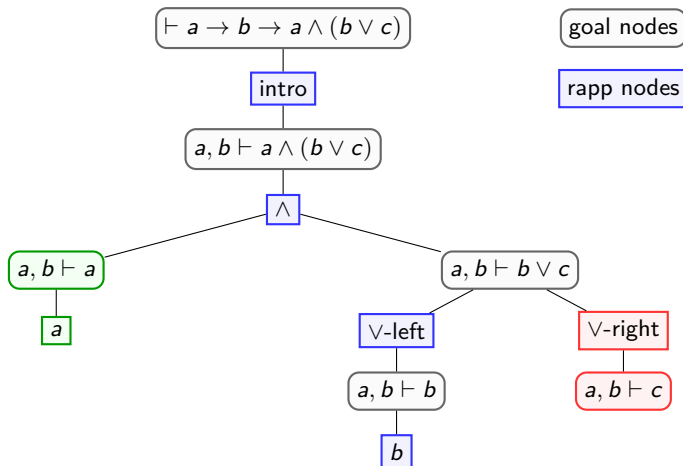


Search tree

proved
stuck
unknown

goal nodes

rapp nodes

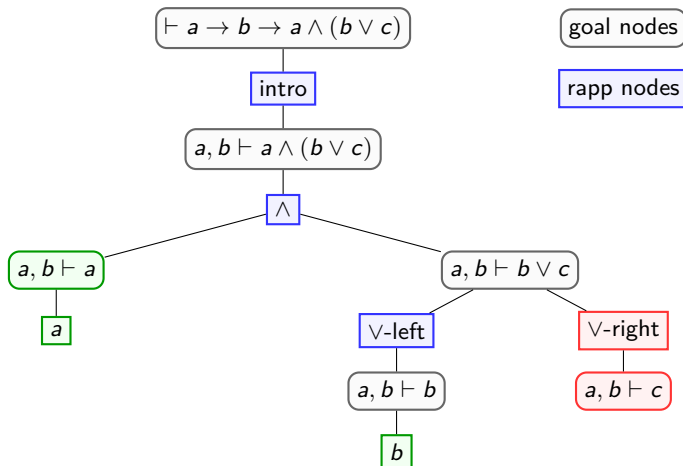


Search tree

proved
stuck
unknown

goal nodes

rapp nodes

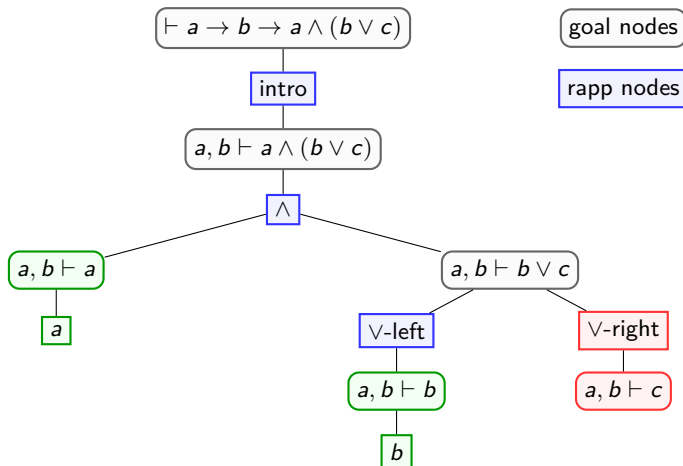


Search tree

proved
stuck
unknown

goal nodes

rapp nodes

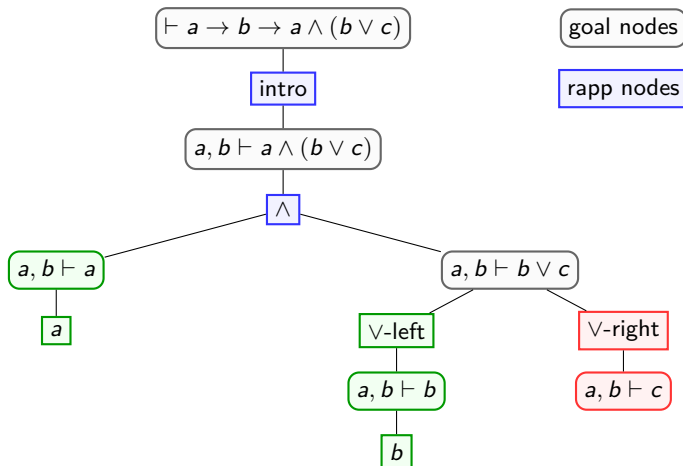


Search tree

proved
stuck
unknown

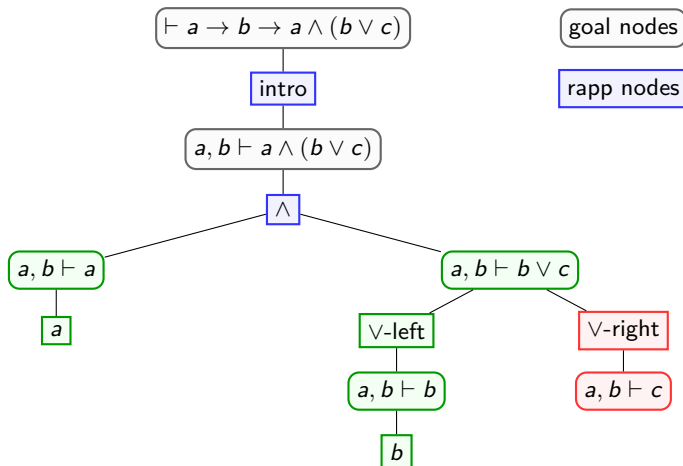
goal nodes

rapp nodes



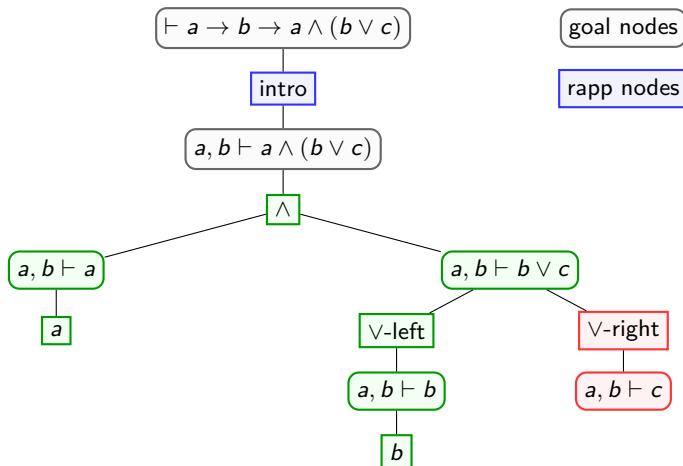
Search tree

proved
stuck
unknown



Search tree

proved
stuck
unknown

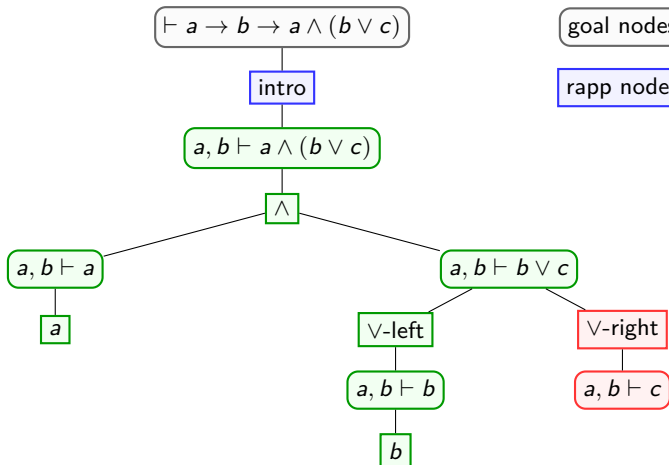


Search tree

proved
stuck
unknown

goal nodes

rapp nodes



Search tree

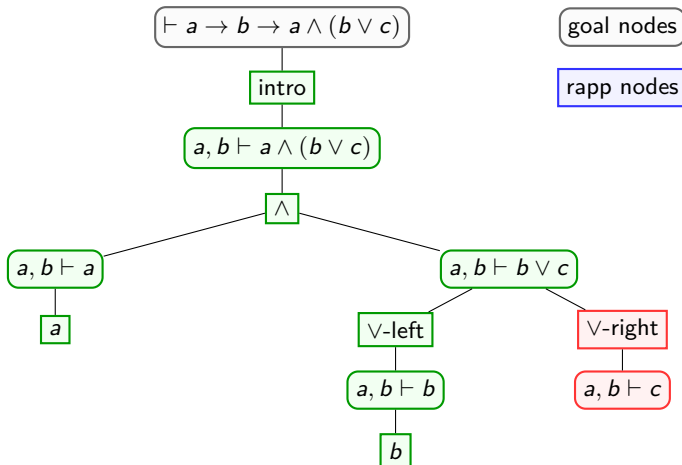
proved

stuck

unknown

goal nodes

rapp nodes

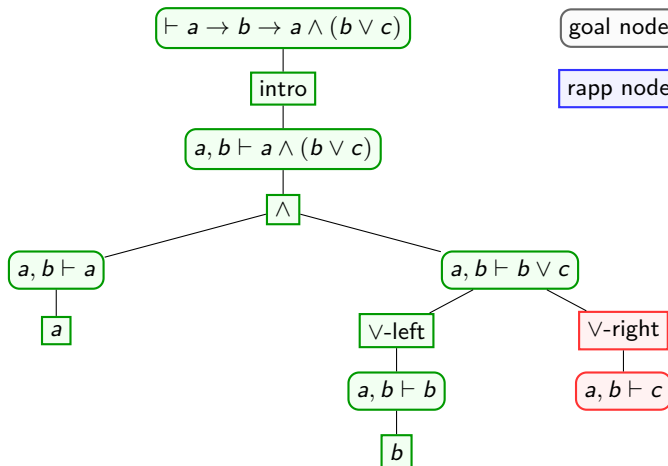


Search tree

proved
stuck
unknown

goal nodes

rapp nodes



Rule distinction

Aesop distinguishes between 3 types of rules:

Rule type	Preserves provability	#Subgoals	Effect
Normalisation	✓	≤ 1	Simplification
Safe Rule	✓	arbitrary	Safe branching
Unsafe Rule	Not necessarily	arbitrary	Subgoals can get stuck

Unsafe rules might require backtracking and an extensive search.

Normalisation rules

Idea & properties

- ▶ Normalise/Simplify the initial goal.
- ▶ Generate ≤ 1 subgoals
→ no branching.
- ▶ Preserve provability.
- ▶ Have a user-specified integer priority
→ determines order.

Typical examples

- ▶ Introduce assumptions,
- ▶ Unfold definitions,
- ▶ Prove trivial equations.

Warning

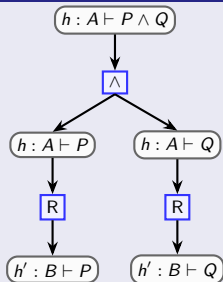
Rules have to be chosen carefully to ensure that they preserve provability.

Safe rules

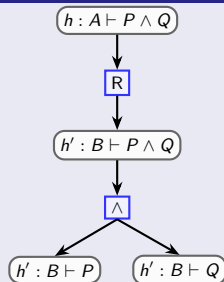
Similar to normalisation rules (incl. integer priority), but there can be more subgoals.

Important: The priority impacts performance! Example: Consider the safe $\wedge$ -rule and another rule R that transforms $h : A$ into $h' : B$.

Bad: Prioritize $\wedge$ -rule



Good: Prioritize rule R

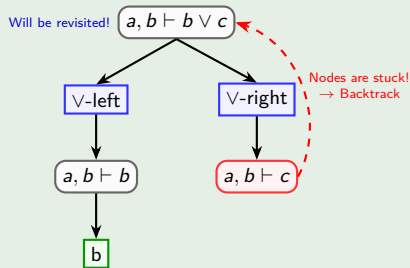


Unsafe rules

Idea & properties

- ▶ Do not necessarily preserve provability.
- ▶ Have a user-specified **success probability** between 0% and 100%.
→ guide the search.
- ▶ Backtracking might be required.

Backtracking example



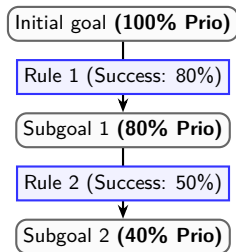
Best-First Strategy: Goal priority

Each goal node has a priority between 0% and 100%.

The computation:

- ▶ Initial goal has a priority of 100%.
- ▶ Applying a rule yields subgoals with the same priority as the parent goal scaled by the success probability

$$\text{Prio}(G_{\text{sub}}) = \text{Prio}(G) \cdot P_{\text{Success}}(R).$$



Best-First Principle

Aesop works with goals of unknown state with the **highest priority**.

Overview of the algorithm

Start with a single goal and go into the following loop:

Main loop

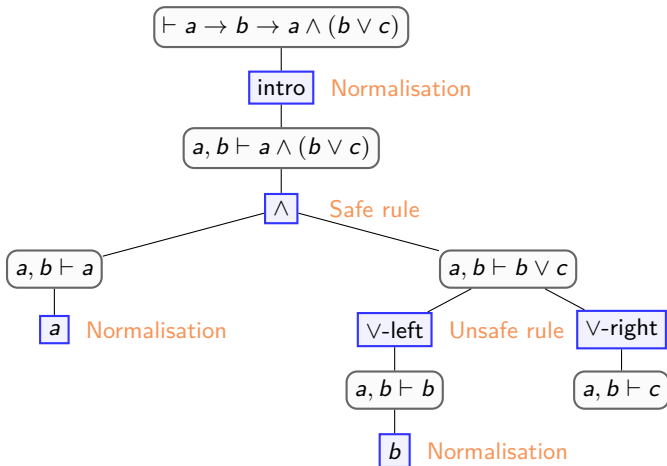
- ▶ Choose an **unknown goal** G with **highest priority**.
- ▶ **Expand** along G with a not yet applied rule as below.
- ▶ Terminate if the initial goal becomes **proved** or **stuck**.

Expansion of G in 3 distinct phases:



- ▶ **Rule applies:** Add subgoals to the tree and start the next iteration.
- ▶ Unsafe rules are processed in order of their success probability.

Example overview

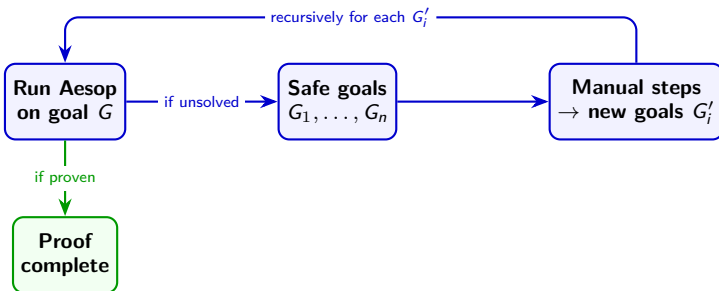


Safe goals

What happens if Aesop fails:

- ▶ Aesop reports the **safe goals**, that remain after using **only** normalisation and safe rules.
- ▶ Indicate how much progress can be made safely.

Typical workflow:



Usage in Lean

Rule builders

Motivation: Aesop rules are arbitrary tactics.

→ inconvenient to write a tactic for each definition/lemma.

Solution: Rule builders

- ▶ Register expressions as Aesop rules.

Registration:

- ▶ **Local:** For a single Aesop call.
- ▶ **Global:** Rules are added to a **rule set**.

Rule sets:

- ▶ Rule sets can easily be activated or deactivated.
- ▶ 2 standard rule sets: **builtin** and **default**.

apply

forward

destruct

constructors

cases

simp

unfold

tactic

default

Rule builder: apply (Backward reasoning)

- ▶ Given: A term f of type $\forall \vec{x} : \vec{T}, P\vec{x}$.
- ▶ Apply builder creates a rule, that applies f to goals $\Gamma \vdash P\vec{y}$.
- ▶ Arguments $\vec{x}$ are inferred or become subgoals.

Example in Lean:

```
@[aesop safe apply]
lemma mp (h1 : a → b) (h2 : a) : b
  := by sorry
```

→ marked as safe rule

Effect in Aesop:

- ▶ Given: Goal $\vdash b$.
- ▶ Aesop applies lemma mp.
- ▶ Two subgoals:
 - ▶ $\vdash a \rightarrow b$
 - ▶ $\vdash a$

Rule builder: forward (Forward reasoning)

- ▶ Given: A term f of type $\forall \vec{x} : \vec{T}, P\vec{x}$.
- ▶ Forward builder creates a rule, that performs forward reasoning with f .
- ▶ If a goal contains hypotheses $\vec{h} : \vec{T}$, the rule adds a new hypothesis $h' : P\vec{h}$.

Example in Lean:

```
@[aesop safe forward]
lemma mp (h1 : a → b) (h2 : a) : b
  := by sorry
```

→ marked as safe rule

Effect in Aesop:

- ▶ Given: Goal $\Gamma \vdash U$ and Γ contains hypotheses
 - ▶ $h1 : a \rightarrow b$
 - ▶ $h2 : a$
- ▶ Aesop uses lemma mp.
- ▶ New hypothesis: $h3 : b$

Rule builder: simp (Normalisation)

- ▶ Aesop uses Lean's simplifier with the default global set of rewrite rules and a separate Aesop rule set.
- ▶ Simp builder creates a rule, that is added to the latter rule set.

Example in Lean:

```
@[aesop norm simp]
lemma add_zero (x : ℕ) : x + 0 = x
  := by sorry
```

→ marked as normalisation rule

Effect in Aesop:

- ▶ Given: Goal

$$\vdash (x + 0) + y = z$$

- ▶ Aesop uses `simp` during normalisation.
- ▶ The goal is simplified to

$$\vdash x + y = z.$$

Rule builder: constructor (Inductive types)

- ▶ Constructor builder creates a rule, which applies the constructors of an inductive type I .
- ▶ The same effect as adding each constructor of I as an apply rule.
- ▶ Here: The apply rules are combined into one **multi-rule**.

Example in Lean:

```
@[aesop unsafe 50% constructors]
inductive MyOr (a b : Prop) where
| inl : a → MyOr a b
| inr : b → MyOr a b
```

→ marked as unsafe rule

Effect in Aesop:

- ▶ Given: Goal $\vdash \text{MyOr } a \ b$.
- ▶ Aesop detects the type and generates subgoals for the constructors.
- ▶ Two paths:
 - ▶ $\vdash a$ (via inl)
 - ▶ $\vdash b$ (via inr)

Adding rules/rule sets

- **Declaration** of rule sets.

```
declare_aesop_rule_sets [r1, ..., rn] (default := <bool>)
```

- **Adding** a rule to Aesop.

```
@[aesop <phase>? <priority>? <builder>? <builder_option>*  
    <rule_sets>?]
```

- <phase> is norm, safe, unsafe.
- <priority> is a natural number, integer priority or percentage between 0% and 100% (depending on the scenario).
- <builder> The rule builder.
- <builder_option> is used for further specifications of the rule builders.
- <rule_sets> are the rule sets to which the rule will be added.

Applying aesop

- **aesop** to use the tactic.

```
example (a b c : Prop) : a → b → a ∧ (b ∨ c) := by
  aesop
```

- **aesop?** returns the proof found by aesop.

```
example (a b c : Prop) : a → b → a ∧ (b ∨ c) := by
  aesop?
```

```
example (a b c : Prop) : a → b → a ∧ (b ∨ c) := by
  intro a_1 a_2
  simp_all only [true_or, and_self]
```

Applying aesop

- ▶ Aesop can be called with more parameters:

```
aesop
  (add safe foo, 10% cases Or, safe cases Empty)
  (erase A, baz)
  (rule_sets := [A, B])
  (config := { maxRuleApplicationDepth := 10 })
```

- ▶ **add/erase**: Adds/Erases rules without affecting rule sets.
- ▶ **rule_sets**: Uses the given rule sets along with *default* and *builtin*.
- ▶ **config**: Configures the search strategy, maximum search depth, maximum applications of normalization rules, [...].
- ▶ **simp_config**: Configures the built-in norm simp rule.

Tracing, Profiling and more

- ▶ Print actions taken by Aesop during the proof search.

```
set_option trace.aesop true
```

- ▶ Once the search has concluded, print the final search tree.

```
set_option trace.aesop.tree true
```

- ▶ If the search is successful, print some statistics.

```
set_option trace.aesop.stats true
```

- ▶ [...]

Including Metavariables

Coupling problem

Transitivity of $\leq$

Given the goal $n \leq k$, assume we use $\leq$ -trans as rule. This creates two subgoals with a shared metavariable $?m$:

$$n \ k : \mathbb{N} \vdash n \leq ?m$$

$$n \ k : \mathbb{N} \vdash ?m \leq k$$

Possible solutions for goal 1:

- ▶ Proof via theorem $\forall n, n \leq n + 1$:
→ Goal 2 becomes $n + 1 \leq k$.
- ▶ Proof via local hypothesis $n \leq a$:
→ Goal 2 becomes $a \leq k$.

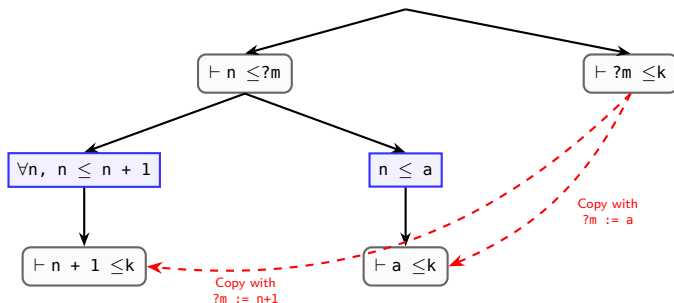
The problem

The subgoals are no longer independent. The way the first goal is proven determines whether the second goal can be closed.

The solution

Instantiations as new goals

Aesop treats goals with changed metavariables as **additional subgoals**. If an assignment is made, the dependent goal is copied along with the assignment into the new proof branch.



Consequences

Large search space

As Aesop copies a large number of goals when dealing with coupled metavariables in order to test all parallel assignments, the search can be slowed down significantly.

Best Practices for Lean

- ▶ **Local registration:** Rules, that generate metavariables, should not be included in the global rule set.
- ▶ **Safe becomes unsafe:** If a *safe*-rule assigns a metavariable, it is treated as an *unsafe* rule (with 90% success probability).

Thank you for your attention!